

torch.jit.set_fusion_strategy#

https://pytorch.org/docs/stable/generated/torch.jit.set_fusion_strategy.html

Description:

Set the type and number of specializations that can occur during fusion.

Usage: provide a list of pairs (type, depth) where type is one of “STATIC” or “DYNAMIC” and depth is an integer. In STATIC fusion, fused ops are compiled to have fixed input shapes. The shape is determined based on some initial profiling runs. In DYNAMIC fusion, fused ops are compiled to have variable input shapes, so that multiple shapes are possible. In both cases, we also recompile on new striding behavior, device, or dtype. When an input doesn’t match the format required by the specialized compiled op, it will run a fallback function. Fallback functions are recursively be compiled and specialized based on the observed tensor shapes. Since compilation can be slow, the “depth” parameter is provided to limit the number of specializations that can be compiled, before giving up on recompiling and falling back to a completely unfused, un-specialized implementation.

Function Signature

```
torch.jit.set_fusion_strategy(strategy)[source]#
```

Behavior - static vs dynamic:

Behavior - fallback functions & depth:

Detailed Documentation

Documentation

Set the type and number of specializations that can occur during fusion.

Usage: provide a list of pairs (type, depth) where type is one of “STATIC” or “DYNAMIC” and depth is an integer.

In STATIC fusion, fused ops are compiled to have fixed input shapes. The shape is determined based on some initial profiling runs. In DYNAMIC fusion, fused ops are compiled to have variable input shapes, so that multiple shapes are possible.

In both cases, we also recompile on new striding behavior, device, or dtype.

When an input doesn't match the format required by the specialized compiled op, it will run a fallback function. Fallback functions are recursively be compiled and specialized based on the observed tensor shapes. Since compilation can be slow, the “depth” parameter is provided to limit the number of specializations that can be compiled, before giving up on recompiling and falling back to a completely un-fused, un-

specialized implementation.

The list of (type, depth) pairs controls the type of specializations and the number of specializations. For example: [(“STATIC”, 2), (“DYNAMIC”, 2)] indicates that the first two specializations will use static fusions, the following two specializations will use dynamic fusion, and any inputs that satisfy none of the 4 options will run an unfused implementation.

NB: in the future, if more as more fusion backends are added there may be more granular apis for specific fusers.